basic application with identifiable weaknesses:

The Campus API application was developed using the following tools:

- Language: JavaScript (ES modules)
- Runtime and server: Node.js, Express
- Containers: Docker, Docker Compose
- Dependency management: pnpm
- Authentication: jsonwebtoken, weak hardcoded secret, JWT with no expiration
- Passwords: Stored in plaintext
- File uploads: Multer with insufficient validation
- Tests: Node.js Test Runner (node:test), HTTP API via fetch
- Deliberately outdated dependencies: Express 4.17.1, jsonwebtoken 8.5.1, Multer 1.4.5-lts.1

The application allows registration, login, access to the profile, and simple REST operations.

The 5 weaknesses of the application (unsecure):

- Password stored in plaintext (clear-text password).
- Weak JWT secret, hardcoded (weak JWT secret).
- Insufficient access control allowing IDOR (Insecure Direct Object Reference).
- Missing rate limiting for login attempts (missing rate limiting).
- File upload without sufficient validation (unvalidated file upload).

Method	Route	Authentication	Function
GET	/health	No	Check that the API is working
POST	/auth/register	No	Create an account
POST	/auth/login	No	Log in, returns a JWT and a cookie
POST	/auth/logout	No	Delete the session cookie
GET	/users/me	Yes	Retrieve your profile
GET	/users/:id	Yes	Retrieve a profile without verifying that it belongs to the logged-in user
POST	/files	Yes	Upload a file without sufficient validation
GET	/files/:id	No	Retrieve a file without access control

Testing the weaknesses:

pnpm --dir unsecure install --frozen-lockfile

pnpm --dir unsecure test

The program verifies that the five weaknesses are indeed present in the vulnerable application: plaintext password, weak JWT, IDOR, missing rate limiting, and unvalidated upload.

Fixed application:

The Campus API application was developed using the following tools:

- Language: JavaScript (ES modules)
- Runtime and server: Node.js, Express
- Containers: Docker, Docker Compose
- Dependency management: pnpm
- Authentication: jsonwebtoken with a secret from the environment, JWT expiration,
- HttpOnly cookie
- Passwords: bcryptjs
- Validation: Zod
- HTTP and CORS protection: Helmet, restricted CORS
- Rate limiting: express-rate-limit
- File uploads: Multer, file-type, temporary quarantine
- Logging: Pino
- Tests: Node.js Test Runner (node:test), HTTP API via fetch
- Dependency audit: npm audit via the security checker

The 5 weaknesses fixed:

- Plaintext password → passwords are hashed with bcryptjs.
- Weak JWT secret → the secret comes from the configuration and must be sufficiently long, JWTs have an expiration.
- IDOR → access to a profile or file is limited to its owner or an administrator.
- Missing rate limiting → login attempts are limited.
- Unvalidated upload → the file size, type, and content are verified.

Additional security mechanisms added:

- File quarantine: files are isolated before validation and moved to the final storage.
- Suspicious activity detection: repeated failed login attempts trigger a temporary lockout and are logged.
- Dependency analysis: the security checker runs npm audit to detect vulnerable dependencies. This is a project-level control, not a protection executed on every request.

Security checker:

- Checks the Node.js version and its update status.
- Checks dependencies for vulnerabilities.

Test report:

Both test suites passed, with no failing tests.

Application	Result
unsecure	9 passed / 9
secure	10 passed / 10

The unsecure tests confirm, plaintext passwords, weak JWT, IDOR, missing rate limiting, and unvalidated upload.

The secure tests verify the corresponding protections, as well as the HttpOnly cookie, quarantine, lockout after repeated failures, and dependency audit. The production audit passed.

Risks to address as a priority:

Problem:
Sessions are not immediately revocable: logout clears the cookie, but an already copied JWT remains valid until it expires. The token is also returned in the body of the login response.

Solution:
Do not return the token if the frontend uses the cookie, add session revocation or rotation.

Problem:
In-memory storage: users, files, and failed attempts disappear on restart, and multi-instance operation is not reliable.

Solution:
Use a database in Docker and durable file storage, with backups. Also move rate limits and lockouts to shared storage.

Problem:
Account lockout can be exploited to block a user: five failures on an identifier trigger a temporary lockout.

Solution:
Adjust the strategy to prevent a third party from easily blocking an account, monitor attempts by IP and by account.

Finally: add the audit and tests to a CI pipeline, monitor development dependencies as well, and run the application on Kubernetes.